Simple Banking System

CSC 1102 Structured and Object-Oriented Programming – Practical Exam Project – Group 24

1. Group details

Name	Student Number	Registration No.
Lubega Michael (Group leader)	2600703548	26/U/03458/PS
Sebalemba Dennis	2600726046	26/U/26046/PS
Muyonjo Solomon	2600703581	26/U/03581/PS
Rob Wilbert	2600726537	26/X/26537/PS
Ssenfuma Emmanuel	2600703644	26/U/03644/PS

GitHub repository: https://github.com/c-eo/oop-practical

2. Project description

The Simple Banking System is a Java Swing desktop application for managing bank accounts. A user can create Savings or Current accounts, deposit and withdraw money, add interest to savings accounts and delete accounts. A table shows every account, the header shows the number of accounts and the total balance, and an activity log records every action. Invalid input (empty names, non-numeric or non-positive amounts, insufficient funds) is rejected with a clear message instead of crashing the program.

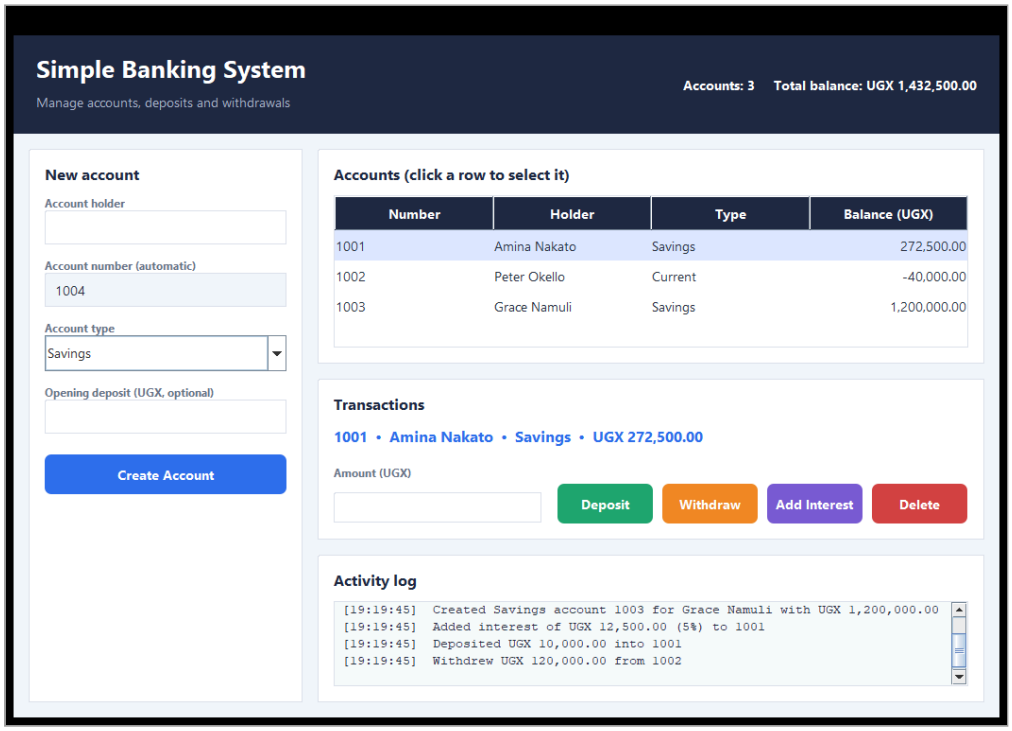


Figure 1: The main window with three accounts and the activity log

3. Classes

Class	Role
`BankAccount`	Abstract base class. Private holder name, final account number and private balance; default and parameterised constructors; getters; a validated

	setter; `deposit()` and `withdraw()`.
`SavingsAccount`	Extends `BankAccount`. Adds a 5% interest rate and `addInterest()`.
`CurrentAccount`	Extends `BankAccount`. Overrides `withdraw()` to allow an overdraft of UGX 50,000.
`InsufficientFundsException`	Custom checked exception thrown when a withdrawal is too large.
`Bank`	Holds a `BankAccount[100]` array and provides add, find, remove, total balance and next account number.
`BankingGUI`	The Swing window: builds the interface, handles button clicks and contains `main()`.

How they work together: `BankingGUI` creates one `Bank`. When the user creates an account, a `switch` on the chosen type builds a `SavingsAccount` or `CurrentAccount`, which is stored in the `Bank` array. After every action the table is rebuilt from that array so the screen always matches the data. Deposit and withdraw are called on the selected `BankAccount`; exceptions thrown by the account classes are caught by the GUI and shown in a dialog.

4. How the exam requirements are met

Requirement	Where in the project
At least 3 classes	Six classes (see section 3)
Constructors	`BankAccount()` (default, calls `this(...)`) and `BankAccount(holder, number, balance)`; subclasses use `super(...)`
Access modifiers	`private` fields, `protected` `setBalance()` and `validateAmount()`, `public` methods
Getters and setters	`getAccountHolder()`, `getBalance()`, `setAccountHolder()` with validation; no public setter for balance or account number
Objects	`Bank`, `SavingsAccount` and `CurrentAccount` objects created in the GUI
Flow control	`if/else` (validation), `switch` (account type), `for` (searching and totals), `while` (`nextAccountNumber()`)
Arrays of objects	`BankAccount[] accounts` in `Bank`
Swing GUI	`BankingGUI`: JFrame, JTable, JComboBox, JTextField, buttons, JOptionPane dialogs
Inheritance / polymorphism	Two subclasses override `getAccountType()`; `CurrentAccount` overrides `withdraw()`; the array holds both types
Exception handling	Custom `InsufficientFundsException` with try/catch in the GUI

5. Key code

Encapsulation and constructors (`BankAccount`): the balance is private and the setter validates.

```
private String accountHolder;
private final String accountNumber;
private double balance;

public BankAccount() { this("Unknown", "0000", 0.0); }

public void setAccountHolder(String accountHolder) {
    if (accountHolder == null || accountHolder.trim().isEmpty())
```

```
        throw new IllegalArgumentException("Account holder name cannot be empty.");
    this.accountHolder = accountHolder.trim();
}
```

Overriding (`CurrentAccount`): the overdraft limit extends what can be withdrawn.

```
@Override
public void withdraw(double amount) throws InsufficientFundsException {
    validateAmount(amount);
    if (amount > getBalance() + overdraftLimit)
        throw new InsufficientFundsException("Overdraft limit exceeded...");
    setBalance(getBalance() - amount);
}
```

Array of objects (`Bank`): removing an account shifts the later ones left.

```
for (int j = i; j < count - 1; j++) accounts[j] = accounts[j + 1];
accounts[count - 1] = null;
count--;
```

6. How to run

Requires a JDK (Java 8 or later). Open a terminal in the project folder and run:

```
javac *.java
java BankingGUI
```

7. Conclusion

The project applies the main ideas of the course in one small program: encapsulation, inheritance, polymorphism, abstraction, exception handling, arrays of objects and a Swing interface. Each class has one clear job, and all the money rules live in the account classes rather than in the GUI, which makes the program easy to read, test and extend.